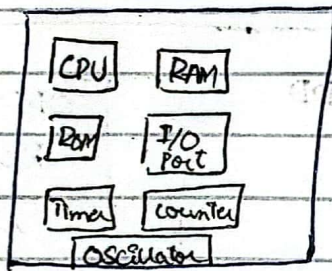


19 Sep

Lecture 9

Microcontroller



within one chip everything is connected

Oscillator provide clock pulse

* Computer is a general purpose device (not use for specific purpose)

μ Processor → General Purpose → more power consumption
↳ more memory

μ Controller → Application Specific

↳ use in embedded systems

↳ certain amount of specific RAM & ROM

Example

↳ Arduino

↳ Raspberry pi

↳ PIC

↳ MSP32

RAM

↳ operand data store

PIC 8 bit microcontroller from microchip
↳ peripheral Interface Controller
↳ Execution is perform in ALU
→ Harvard Architecture (separate RAM + ROM)

→ RISC architecture

↳ same size instruction

↳ fewer instruction (code length is increase but instruction cycle is smooth)

Assembly is different for different architecture

Instruction Set

↳ CISC

↳ RISC

Size of instruction ← CISC → complex instruction set computing

vary

← Size of instruction ← RISC → reduce instruction set computing

remain same

Easy to write & compile code
Benefit

8086 → CISC

↳ No of instruction known

↳ but complex

↳ instruction size vary

EEPROM

↳ type of ROM

↳ Erasable

↳ Extra amount of

memory require

so we can utilize

OSC → oscillator

↳ clock provide

Port

Basic Architecture
(8 bit)

WREG register

↳ working register

temporary register (Koi operation perform Karne to operand
edge store hoga)

1 cycle → 8 bit

2 cycle → 16 bit [lower bit → 1 cycle
higher bit → 2 cycle

MOV LW (move command) →

↓
literal working register

↓
operand (SS, SB)

literal ko working register me move kar dena ha

MOL LW 87H (Load 87H into WREG)

ADD LW Instruction

MOV LW 55H
Add LW 02H

(Ans 57H store in working register)

* 2 1 bit add max
result 3 bit

* 2 8 bit add max
result 9 bit

9 bit store in carry flag

Lecture 10

26 September 2024

1 chip → all things

PIC → 8 bit

→ Harvard

→ separate

RAM + ROM

→ RISC

(same size of instruction)

Complexity reduce

PIC

RAM & also known as File Register

RAM divided into 2 types of register

→ data & store

RAM / File Register

SFR

Special function register

→ dedicated
→ specified
→ stores nothing

→ interrupt, timer etc memory allocate karta wo change nahi ho sakti

Family of PIC Register

PIC 12

PIC 18

↓

base version

↓

Higher

GPR (like Ax, Bx in 8086)

General purpose register

→ open (can store any data of our choice)

→ programmer ko access hai

8086

PIC

Operand

Literal

Operand = Literal

→ SFR barely changes

→ GPR continuously change

→ GPR store changes along with it

→ GPR (max position)

→ SFR (min position) allocated

→ EEPROM

(Electrical Erasable)
(extra memory)

Anything we are storing is on GPR

Store kar raha working register ka litent ko file register (RAM) store karla

3) MOV WF → File register (it can be even 1 bit & byte)

↓

working register
(8 bit)

Working register

→ 8 bit

File register

→ bytes ma bi ho sakte

Working Register value is move in file

PORT B

I/O Port → SFR

MOV Wf (Location)

Konsi dex
copy karne
ko yahan

pham move
karwane
ko yahan par

PORT B }
PORT C } SFR
PORT D }

IO PORT } SFR

MOV Wf ka baad operand / literal nahi aay ga Location aay gi yahan hum copy kar
rahe hn

4) ADD Wf

(add working register + File register)
working register ma value ka

→ working register value already
→ File register nahi pata wo specify
karne

File specify nahi

ADD Wf Location in File reg, D

→ destination (when perform arithmetic
operation so it is store in working or
file register)

ADD Wf 55H, 0H 0 zero (working register ma store hoga)

ADD Wf 55H, 1H 1 one (file register ma store hoga)

ADD Wf 55H, W W (working register ma store hoga)

ADD Wf 55H, F F (file register ma store hoga)

Code

MOV LW 22H (working register ma 22 move)

MOV Wf 5H

MOV Wf 6H

MOV Wf 7H

5, 6, 7 Location par 22 store
hoga

05	22
06	22
07	22

ADD Wf 5H, 0 (22 + 5 add kar ka working register ma store

ADD Wf 6H, 0 kar do

ADD Wf 7H, 0

WREG = 44

WREG = 6H

WREG = 88H

ADD WF 7, 1

mean working register 66H

file register 88H at 7 done

Code

MOV LW 0

WREG = 0 (1 instruction)

MOV WF 12H

WREG = 22 (3 instructions)

MOV LW 22H

ADD WF 12H, F (22+0)

12	22
----	----

ADD WF 12H, F (22+22)

12	44
----	----

ADD WF 12H, F (22+44)

12	66
----	----

ADD WF 12H, F (22+66)

12	88
----	----

MOV LW 0 (WREG = 0)

WREG = 0

MOV WF 12H (move 0 at 12H)

MOVLW 22H

ADDWF 12H, W (0+22)

12H	0
-----	---

ADDWF 12H, W (22+22)

ADDWF 12H, W (22+44)

ADDWF 12H, W (22+66)

F → We need to specify RAM Location

COMF (complement and store in file)

GOTO (unconditional Jump) (continuously Karay raho)

GOTO (never ending loop without any instructions)

MOV LW 3

MOV WF 20H

3 → 2 → 1 → 0

DEC F 0x20, F (2)

store in 20H

DEC F 0x20, F (1)

DEC F 0x20, F (0)

hexadecimal
writing way (address)

5) MOV F instruction

MOV F → MOV FW

6) MOV FF

(port B ke value port C sa move kar do)

PB → W → PC] MOV F

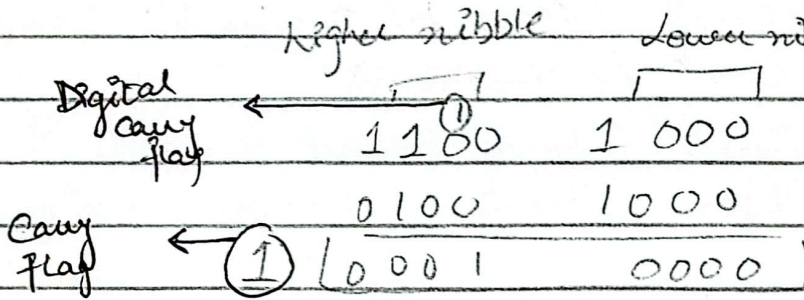
PB → PC] ~~MOV FF~~ (preferable
F as it is copy)

→ If we want to perform any
arithmetic or processing
then it is preferable

C → Carry flag

DC → Digital Carry flag

↳ Auxiliary carry (16 bit no → word)
(8 bit → byte)
(4 bit → Nibble)



Auxiliary / Digital

Carry from Lower nibble to Higher nibble

Zero flag (Jo last operation perform Kiya aurka result Zero ka equal to Zero flag set (1) ho jayega)

Overflow flag

Negative flag (jab -ve value aaygi to wo)

1 Oct 2024

Lecture 11

Example 2.8

MOV LW 38H

ADD LW 2FH

38H

2FH

67H

0011 1000

0010 1111

0110 0111

DC → 1 (lower bit to upper bit transfer)

C → 0 (no final carry)

Z → 0 (puray answer na agai)

kar bit 0 to zero flag 1 hoga

Zero flag = 0 (non zero value)

Example 2.10

MOV LW 88H

ADD LW 93H

88H

93H

1000 1000

1001 0011

10001 1011
carry

Carry flag → 1

DC → 0

Zero flag → 0

PIC

↳ Jump is known as branch

Branching are on the bases of flag

BC → Check carry flag

BNC → Check carry flag is not 1

* flag use to observe

all (complete)

result of add

& sub

* Default

data type

↳ hexa

Microcontroller ki memory interfacing kaise karate hai (as it is connected in embedded system)

000H
 000
 4 4 4
 12 bit

PIC ROM 4KB $2^2 + 2^{10} = 2^{12} = 12 \text{ bit}$

Starting address 000H

12 bit address bus

Ending address FFFH

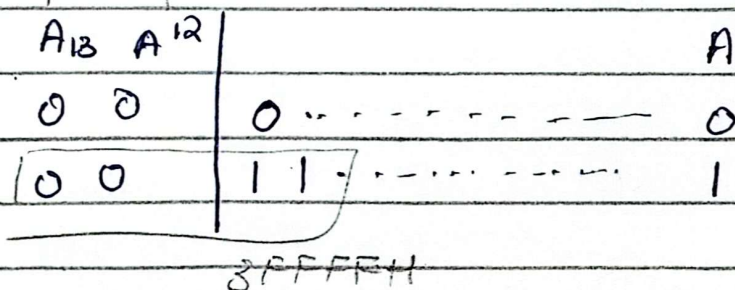
0000H
 16 bit

PIC with ROM 16KB $2^4 + 2^{10} = 2^{14} = 14 \text{ bit}$

Starting address 0000H
 not used

$A_{13} - A_0$

$\rightarrow A_{12} A_{13}$ (is not used)



$\rightarrow$ Extra Bit

$\rightarrow$ zero

$\rightarrow$ Starting address of all memory are zero

PIC with ROM 64KB $2^6 + 2^{10} = 2^{16} = 16 \text{ bit}$

Starting address: 0000H

Ending address: FFFFH

7981
ROM with 128KB

$$2^7 + 2^{10} = 2^{17} \text{ 17 bit}$$

Starting address 00000H

A₁₉ — A₀

A₁₉ A₁₈ A₁₇

A₀

8X16 M1T

Benefit user ko na kuch change kara karo aise hi forget kar delay kr

EQU (Equate)

→ Equal to constant for me hum value store kar sakte hai

Memory Operand

Count EQU 0x25

Label

25H (value store kr lo)

ab we don't know ka koi label ya address

MOV LW Count

25H (Literal)

* Working register
me Literal move kr

MOVLW 0

MOVWF 12H

MOVLW 22H

ADDWF 12H, F

ADDWF 12H, F

Secture 12

Pg 92

Boat 2024

Instruction

CISC $\rightarrow$ 1-6 byte

RISC $\rightarrow$ same size

PIC $\rightarrow$ 2 byte ya 4 byte
 $\downarrow$
 16 bit

MOV LW 25H

(after execution it point to 2 location (0-1 word with execution))

Karay ga aik ma opcode to literal doosray ma lega

(as min size of instruction is 2 byte)

if (4 byte instruction then point after 4)

* Instruction pointer
 jo execute hone wali
 aur ka point karke

* Instruction ka opcode
 generate hoga
 $\rightarrow$ decode hoga or
 opcode execution unit
 ka milke

GOTO } These are not 0 byte

MOV FF } These are 4 byte (32 bit division)

MOV LW Instruction formation (2 byte $\rightarrow$ 16 bit)

higher bit $\rightarrow$ store opcode (8 bit)
 lower bit $\rightarrow$ store literal (8 bit)

MOV LW 25H

0E 25H

ADDV LW 25H

0F 25H

2 instructions ka opcode kabi
 same nahi ho sakta

MOV FF Instruction formation

MOV FF

PORTB

PORTC

Source

Destination

Source & destination

dono ka opcode long

16	16
opcode for source	opcode for destination

RISC → architecture 16 features 19 (100) 8+ on ...
 Harvard architecture 79 on word

Section 5.1

Arithmetic Instruction

(Pg 172)

↳ Signed Number

↳ Unsigned Number

Signed

Instruction 1

Add Lw k

WREG = WREG + k

↳ Literal is specifying

Instruction 2

Add Wf

↳ Location is specifying

IMP

Example 5.2

40 = 7D

41 = EB

42 = C5

43 = 8B

6 → lower byte
 7 → higher byte } Sum

A → 10

B → 11

C → 12

D → 13

E → 14

F → 15

main carry

1 7D

→ D + B = 13 + 11 = 24

D + B = 13 + 11 = 24

EB

24 - 16 = 8

1 68

Spill carry bit
 number is entering
 digit

main carry will generate
 higher byte 1 + 2 = 2

C5

2 D

→ 1 + 7 + E = 22

5B

22 - 16 = 6

Answer = 288

88

→ 8 + 5 = 13 = D

↳ store
 in lower
 byte

→ 6 + C = 12 + 16 = 18

18 - 16 = 2

Example 5.3

3C E7H

3B8DH

16 bit number

Higher 8 bit Lower 8 bit

3C E7 H

3B 8D H

Working register wof 8 bit

16 bit addition



ADDWFC

add auxiliary carry also

use addwfc



add 8bit kar

Kar Ka Karayga

MUL

PRODH → SFR → result of product is store

8 bit registers [PRODL → lower] unsigned
[PRODH → higher]

For division

↳ perform repeated subtraction

$$10 \div 2$$

$$10 - 2 = 8$$

$$8 - 2 = 6$$

$$6 - 2 = 4$$

$$4 - 2 = 2$$

$$2 - 2 = 0$$

with every subtraction

use count a increment

Karay ga

tab tak Karay ga jab tak

jis se minus kar rahay hain us ka kam na a jata

Signed Number

Sign flag in pic $\rightarrow$ negative flag $\boxed{N}$
 $\hookrightarrow D7$

$D7$ ki jo value hogi wo sign flag ki hogi

Agar 1 $\rightarrow N=1$ (-ve number)

0 $\rightarrow N=0$ (+ve number)

Number ko negative ma convert 2's complement.

Overflow

$\hookrightarrow$ always generate in sign arithmetic like sign addition, subtraction

	0000	0000	MSB	LSB
+ve number	0	01	0000	
+ve number	0	01	0010	
-ve number	1	00	0010	

if MSB 1 $\rightarrow$ -ve

$\hookrightarrow N=1$

$OV=1$ (overflow flag set 1) (this tell result is faulty)

$\hookrightarrow$ memory se zada data a gaya $\hookrightarrow$

$D7$	$D6$	$D0$	
-128	-----	127	Decimal
-80		7F	Hexa

128 = 80
129 = 81
130 = 82

If number lie b/w this range overflow flag is 0

if lie beyond this then Overflow flag is 1

to solve this problem we take 2's complement

Condition of generation of overflow flag

→ D₆ sa D₇ par cary ga raha overflow flag = 1

Condition of generation of overflow flag

→ D6 sa D7 par carry ga raha overflow flag = 1

Lecture: 13

8 Oct 2024

Section 5.3

Logic and compare instructions

Logic instructions

↳ AND (jahan zero wahan result clear) (jo bit same rakhnay wahan 1 ka saath masking karway ga)

AND LW 0x0F
literal

LW → Literal value

↳ Literal with working

register

WF → location

↳ OR (use for set desired no of bits)

0x0F

0000

0100

Set → OR

0011

0000

Clear → AND

0011

0100

OR
= $\begin{array}{r} 01001001 \\ 10000001 \\ \hline 11001001 \end{array}$

AND
= $\begin{array}{r} 01001001 \\ 0 \\ \hline 0 \end{array}$

use case 1+2

3, XOR (toggle bit, para koi sakday kis register me konsi value rakhe la)

0x12 1001

XOR

0000

1001

0000

1001

XOR working

Same → output zero

Different → output one

this doesn't give actual result just
tells that number's ka XOR diya wo
opposite ka

11001010

00110011

00110011

11111001

compare

11001010

result of this number ka sath XOR
kiya aur ko compare kar ka
number known ho jayega

Ports (use to communicate with external devices)

↳ SFR

↳ memory consume karta hai

TRIS

Tri State

CLRF TRIS B (port B ko output bana diya)

↳ output ban raha hai

↳ Clear kar raha hai

SETF TRIS B

↳ input bana raha hai

↳ Set 1 kar raha hai

TRIS (A, B, C, D)
 Set to 1/0 port

Example 5.21

read & test Port B (input)

CLRF TRISC (input output bana raha) (CLR output)

CLRF PORTC (port C par? zero hogai)

SEFF TRISB (SF? input)

MOVLW 0x0F

XORWF PORTB, W

Branch Jump ← BNZ EXIT

MOVLW 0x00

MOVWF PORTC

* jahan data bech rahi hai
 wo output hai
 * Port B se bech chuki

XOR Use

↳ Toggle

↳ To find out unknown

↳ Compare

XOR

↳ Selected bit
 toggle karta

COMF (complement fileReg) (1's complement)

↳ puray number ko toggle karta

↳ file register → location

NEG F (2's complement)

PC18 Compare Instructions

CPFSGT (compare file register with working register and if condition fulfilled then skip to next instruction)

C

D '27'
→ Decimal

H '27'
→ Hexadecimal

F XOR

27 > 54

Instruction skip rahi hogi

Page 191

Example 5.24

If XOR to some number? If zero flag generated then

BRA (unconditional jump)

Example 5.26

```
SETF TRISD
CLCF TRISA
CLCF Port A
```

$T = 75$ $wreg = 75$
 $T > 75$ $wreg = T$
 $T < 75$ $wreg = T$

```
MOVLW Temperature
MOVWF wreg
XOR wreg, 75H
CPFSGT
```

```
BCT ( oil level register) (set bit to output Kanca)
BSF
```

10 Oct 2024

Lecture 14

Section 5.4

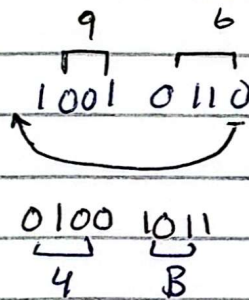
Rotate Instructions and Data Serialization

Rotating without carry:

Pg 195

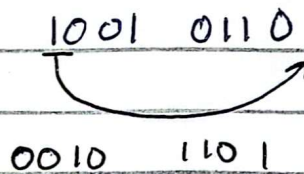
⇒ RRNCF → File
 ↓ ↓ ↘ No carry
 rotate right

(LSB bit rotate become MSB)



⇒ RLNCF

(MSB rotate and become LSB)



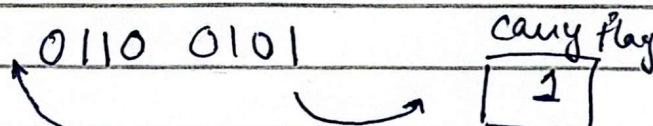
PIC → RISC → instruction size same

↳ instruction with constant size

↳ jitni dafa instruction likhi hotei unhi dafa operation perform hotei

Rotating with carry:

RRCF



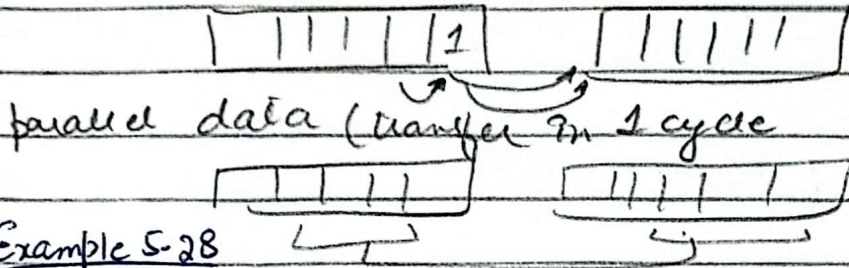
RLCF for reg, d

RLCF 20H, F

↳ meoan 01101101
 store karke value

carry flag ma jo
 carry flag ko check karna number ho wo MSB
 ka wo set ha ya clear hu a gahay ga

Serializing Data (Bit by bit transfer karta)



IMP Example 5-28

RBI
(port B 1 pin)

41H → 8 bit

↳ 8 cycle require
↳ Loop runs 8 time

BCF TRISB, 1 (first pin ko output ban rahi)

MOVLW

MOVWF

BCF STATUS, C (Carry flag ko clear kar rahi)

Flag Register

↳ Status Register

(Haste karna ar liya C ko clear kar rahi)

BSF PORTB, 1 (serially transmit kar rahi)

⇒ this mean storing at MYREG

Again RRCF MYREG(F) (rotate right with carry)

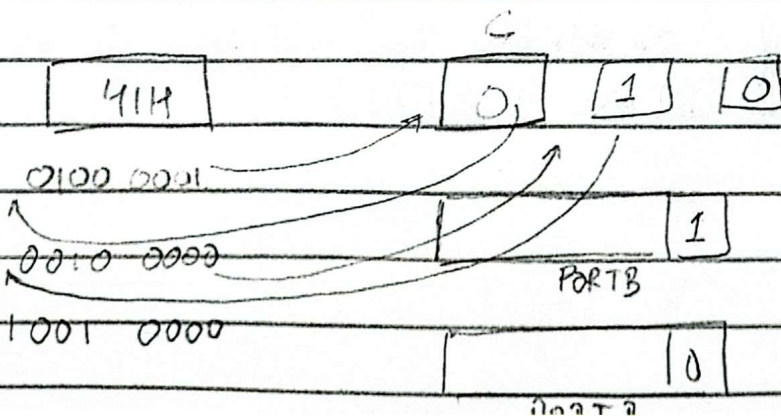
BNC (branch if no carry)

if carry = 0

branch not carry
over BCF PORTB, 1

if carry
then BSF

Logic



PORTB


```

NEXT    DECF    RCNT, F
        BNZ     ] Loop
        BSF

```

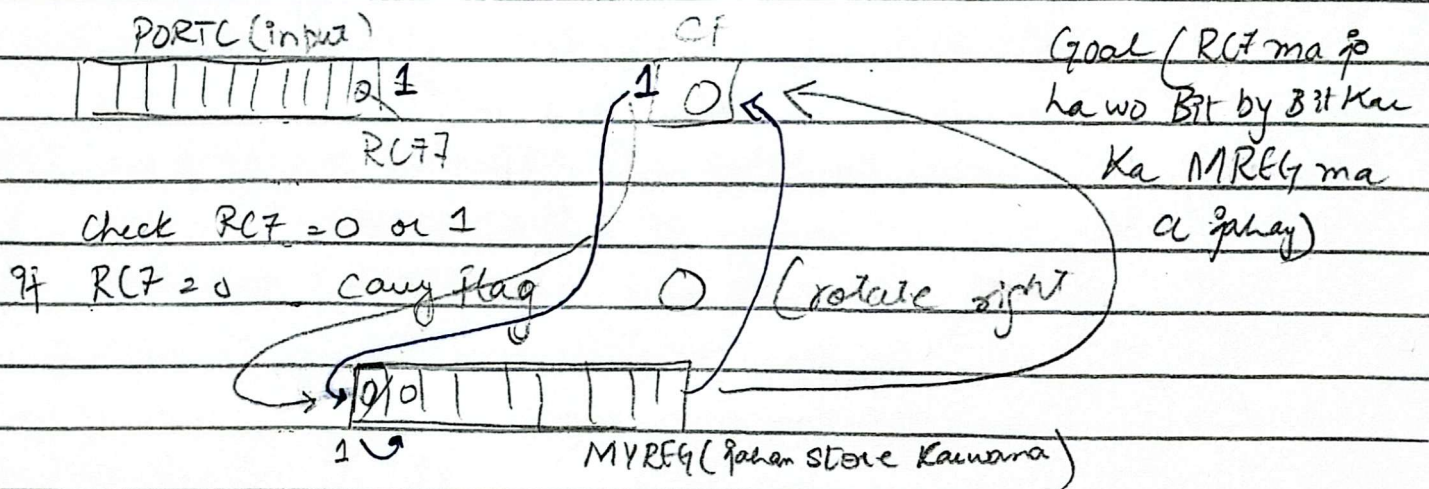
Example 5-2a LSB first (rotate right)

```

BSF     TRISC, 7    (7 pin ko input)

BTFSB   PORTC, 7    (skip next instruction if RC7 = 0)
BSF
BTFSB   PORTC, 7    (skip next if RC7 = 0)
BCF     STATUS, 1    (carry flag clear)
RRCF    MREG, 1      (MREG one time rotate right karna hain)

```



IMP MID QUESTION

If PORTC (whole is transfer to MYREG then we have to rotate PORTC also) (here BTFSB & BTFSB)

15 Oct 2024

Lecture 15

SWAPF

18H

0001 : 1000
HN : LN
1000 0001

1110 0100
0010 0111
Reverse and
Store in
another register

RLNCF

1110 0100 (1)

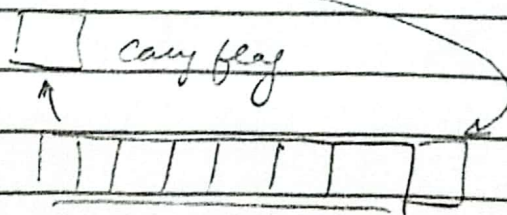
1100 1001 (2)

1001 0011 (3)

0010 0111

REG = 011010

RPCF REG
RLCF REG2



if RLCF
ke flag
RRCF
Same
no back

Reverse of NO

Location

REG2

REG EQU 05H
REG2 EQU 06H
COUNT EQU 07H
MOV LW 05H
MOV WF REG
MOV LW 06H

MOV WF REG2
MOV LW 06H
MOV WF COUNT
AGAIN: RRCF REG
RLCF REG2

DECF COUNT
BNZ AGAIN
END

Palindrome (reverse Karay Ka baad bi same hon)

↳ done by XOR

↳ Subtract

Calculate 2^5

$2 \times 2 \times 2 \times 2 \times 2$

Loop

~~MOV LW 02H~~

~~MOV LW 02H~~

⇒ MOVWF 02H REG

~~COUNT 1 05H~~

~~MULWF~~

~~DECF COUNT~~

~~BNZ loop~~

~~END~~

REG EQU 05H

COUNT EQU 06H

MOV LW 02H

MOV WF REG

loop

MOV LW 02H

MOV LW 02H

MOV WF COUNT ✓

MUL WF REG, F

DECF COUNT

BNZ loop

END

MSD IMP

Q Compare 3 number find 2 Largest

	Num1	Num2	Num2
	X	Y	Z
REG EQU 03H			
REG1 EQU 06H			
REG2 EQU 07H			
MOV LW 02H	Smallest		
MOV WF REG	Largest		
MOV WW 03H	Second largest	equ 0x7	
MOV WF REG1	MOV RW 00H		
MOV LW 04H	MOV WF	Sec-Layer	
MOV WF REG2	MOV FW	Num1.	
	CPFSL	NUM1	
	BRA	NEXT-NUMBER	
	BRA	N-1	
	N-1	(Smallest < Sec-Layer)	
	MOV FW	Smallest	
	CPFSGT	NUM1	
	BRA	Next Number	
	MOV FF	Num1, sec-Largest	
	END		
	NEXT Number		
	MOV FW	Largest	
	CPFSL	NUM2	
	BRA	FINAL	
	BRA	N-2	
	FINAL		
	MOV FF	Num2, sec-Largest	
	END		
	N-2		
	MOV FW	Smallest	
	CPFSG	NUM2	

BRA Final
MOV FF NUM 2, sec-Layer
END

Loop run 300 times

LOOP1 EQU 0x7

LOOP2 EQU 0x8

MOV LW 0x4 (10 move kila ha)

MOVWF LOOP1

MOVLW 0x30

MOVWF LOOP2

MOVLW 0x4

MOVWF PORTB

10 times

MOVLW 0x30

MOVWF LOOP2

30 times

COMF PORTB, F

DEC F LOOP2

BNZ 30 times

DEC F LOOP1

BNZ 10 times

END

17 is multiple of 3

YES → Multiple
NO → Not Multiple
Number move Kar

NUM EQU 0x5

YES EQU 0x6

NO EQU 0x7

MOVLW D'17'

MOVWF NUM

MOVLW D'3'

Loop

SUBWF NUM, F (F-W)

CMPGT NUM

BRA CHECK

BRA Loop

Check:

If > 3 Yes

If < 3 No

24 Oct

Lecture 16

Section 3.2 Call Instructions and Stack

Stack

↳ To store the address of instructions

Purpose

Subroutine (over and over call karnay ki kavirat pahochi)

↳ we can call them routinely

for example we create a module of delay and call it again and again using 'call' instruction

↳ it is also a block

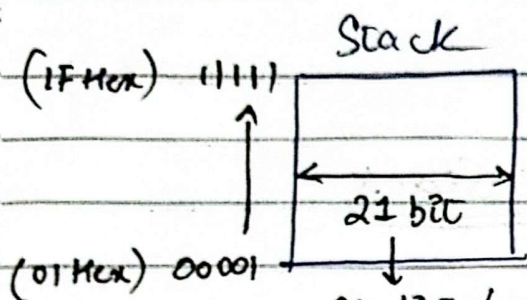
CALL } Difference b/w GOTO & BRA (same as call and RCALL } RCALL } Recall difference!

Long jump Short jump
↓ ↓
4 Byte 2 Byte

PIC Instructions [2 Byte
 4 Byte

CALL → 2 Byte } function is same to call
RCALL → 4 Byte } Subroutine

Return → Subroutine se wapas ana



STACK
↓
LIFO

21 bit long instruction a sakti ha

↳ we are saving instruction address

↳ 31 maximum function address

Push we are pushing the address of instruction into stack
↳ Stack ka andar bhejna

Pop we are retrieving the instruction from stack
↳ Stack se bahar nikalna

Example 3.9

Pg 128

PORT B ko assign
karne se phir
delay instruction
karna

ORG 0 (starting memory address)
Back MOVLW 0x55
MOVWF PORT B

CALL Delay (instruction pointer stop all other instruction and move to delay)

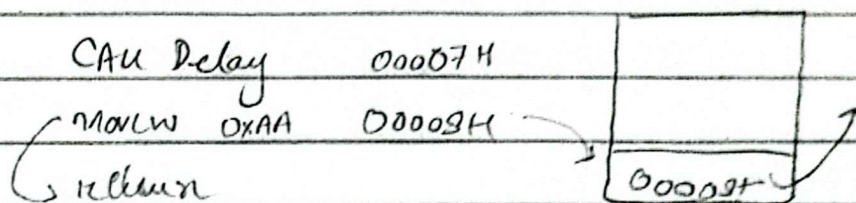
MOVLW 0x55 (after call this instruction ka address stack
MOVWF PORT B ma chala gaya) (return after delay hai)

CALL Delay
GOTO Back

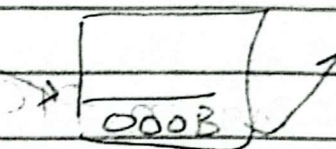
Delay MOV
A Gain

Return (check karay ga stack ma kya pusha ha stack
se wo pop karay ga)

jab return aya stack ma chek karay ga ka kya address push kr aya address per jo instruction wo pop karay ga



CALL Delay
GOTO



Question → Stack ki after call & before call

→ Subroutine

→ Nested Subroutine

we can call 31 nested subroutine

Pg 130

At a time stack ma aik hi subroutine hogi

Pg 131

BACK CALL DISPLAY

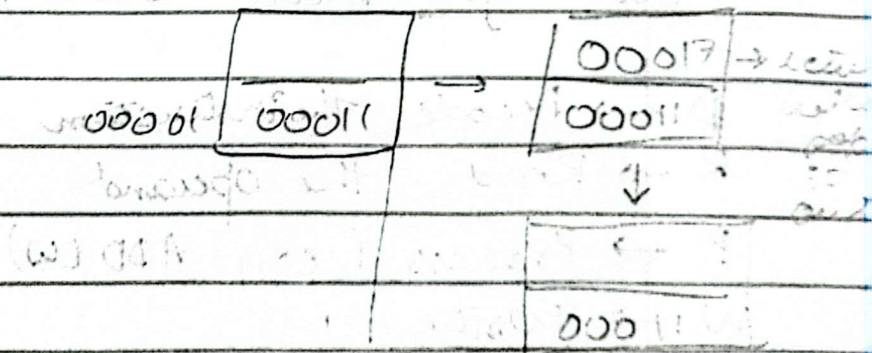
00011 GOTO

DISPLAY

CALL DISPLAY

00017

Return to 00017



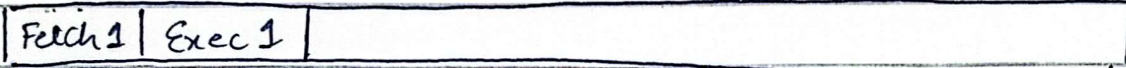
SP → Increment
Instruction pointer

Stack overflow (return sap tara nahi karay ga)

Stack is filling when we are calling nested subroutine to avoid this we don't call nested subroutine

Stack const memory $\rightarrow$ 31 byte

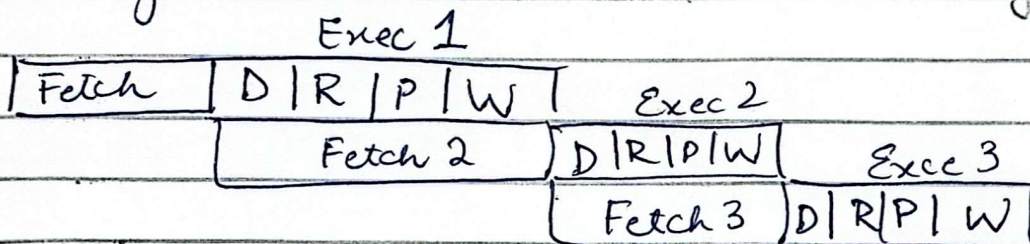
Non pipeline



Pipelining in PIC

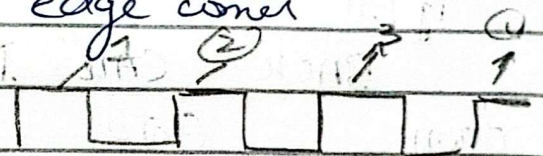
Instruction cycle 1

Clock cycle = 4



In pic microcontroller 1 instruction cycle is completed in 4 clock cycle mean 4 times positive edge comes

D $\rightarrow$ Decode the instruction
R $\rightarrow$ Read the operand
P $\rightarrow$ Process (e.g. ADD LW)
W $\rightarrow$ Write



Disadvantage if branch condition is true then discard instruction the

PIC → 21 bit address lines (2 million)

RCAU → more efficient save 2 byte
within 2KB (RCAU)
Beyond 2KB (CAU)

BANK SELECT → Set a file registers
When all of the memory is not accessible b/c of 8 bit so RAM is
divided into bank

File Register → RAM

↳ SFR (special function register)

↳ GPR (general purpose register)

4KB of RAM

INCF FileReg, d

INCF COUNT, F

↳ 8 bit file register

MOVWF COUNT, MYREG XX

FF → 255

File register

↓
8 bit

	0	1	2	3	...	16
0						
	256 bit	256 bit	256 bit			
255						

16 time 256 is 4KB

Default Bank → 0


```

COUNT EQU 0x8
MOVLW 55H
MOVWF COUNT

```

} Default Bank

```

COUNT EQU 0x8
MOVLW 55H
MOVWF COUNT, 2

```

} Bank 100, 8 Local

Pg 256

MOVLB (literal into bank select register)

In PIC all registers are 8 bit

BSR (bank select register)

